

Numerical Computing

Assignment 6

Topic: Newton Interpolation Polynomial

Name: Saily Kadam

Roll No: MS 2511

Q1 — Building Newton's Polynomial

$$f(x) = x^2 + 2x + 1$$

five data points:

x	-2	-1	0	1	2
f(x)	1	0	1	4	9

1. Theory

1.1 Newton Interpolation

For $n + 1$ data points $(x_0, f(x_0)), (x_1, f(x_1)), \dots, (x_n, f(x_n))$, the Newton interpolation polynomial is written as:

$$P_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \dots + a_n(x - x_0)(x - x_1) \dots (x - x_{n-1})$$

The general divided-difference formula is:

$$f[x_i, \dots, x_{i+j}] = (f[x_{i+1}, \dots, x_{i+j}] - f[x_i, \dots, x_{i+j-1}]) / (x_{i+j} - x_i)$$

2. Given Data

The function is $f(x) = x^2 + 2x + 1$.

$$f(-2) = 4 - 4 + 1 = 1$$

$$f(-1) = 1 - 2 + 1 = 0$$

$$f(0) = 1$$

$$f(1) = 1 + 2 + 1 = 4$$

$$f(2) = 4 + 4 + 1 = 9$$

3. Q1(a) — Divided-Difference Table

3.1 First-Order Divided Differences

$$f[x_i, x_{i+1}] = (f(x_{i+1}) - f(x_i)) / (x_{i+1} - x_i)$$

$$f[-2, -1] = (0 - 1) / 1 = -1$$

$$f[-1, 0] = (1 - 0) / 1 = 1$$

$$f[0, 1] = (4 - 1) / 1 = 3$$

$$f[1, 2] = (9 - 4) / 1 = 5$$

Hence, the first-order divided differences are: **-1, 1, 3, 5.**

3.2 Second-Order Divided Differences

$$f[-2, -1, 0] = (1 - (-1))/2 = 1$$

$$f[-1, 0, 1] = (3 - 1)/2 = 1$$

$$f[0, 1, 2] = (5 - 3)/2 = 1$$

Hence, the second-order divided differences are: **1, 1, 1.**

3.3 Third-Order Divided Differences

$$f[-2, -1, 0, 1] = (1 - 1)/3 = 0$$

$$f[-1, 0, 1, 2] = (1 - 1)/3 = 0$$

Hence, the third-order divided differences are: **0, 0.**

3.4 Fourth-Order Divided Difference

$$f[-2, -1, 0, 1, 2] = (0 - 0)/4 = 0.$$

3.5 Complete Divided-Difference Table

x_i	$f[x_i]$	1st DD	2nd DD	3rd DD	4th DD
-2	1	-1	1	0	0
-1	0	1	1	0	
0	1	3	1		
1	4	5			
2	9				

In matrix form, the Python program produces the following divided-difference matrix:

```

Divided-Difference Table:
[[ 1. -1.  1.  0.  0.]
 [ 0.  1.  1.  0.  0.]
 [ 1.  3.  1.  0.  0.]
 [ 4.  5.  0.  0.  0.]
 [ 9.  0.  0.  0.  0.]]

```

4. Q1(b) — Newton Interpolation Polynomial

The Newton interpolation polynomial for five points is:

$$P_4(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + f[x_0, x_1, x_2, x_3](x - x_0)(x - x_1)(x - x_2) + f[x_0, x_1, x_2, x_3, x_4](x - x_0)(x - x_1)(x - x_2)(x - x_3)$$

Here, $x_0 = -2$, $x_1 = -1$, $x_2 = 0$, $x_3 = 1$.

Newton coefficients are:

$$a_0 = 1, \quad a_1 = -1, \quad a_2 = 1, \quad a_3 = 0, \quad a_4 = 0$$

Therefore:

$$P_4(x) = 1 - 1(x+2) + 1(x+2)(x+1) + 0(x+2)(x+1)x + 0(x+2)(x+1)x(x-1)$$

The zero terms disappear:

$$P_4(x) = 1 - (x+2) + (x+2)(x+1)$$

5. Q1(c) Simplify and Compare

Starting with $P_4(x) = 1 - (x+2) + (x+2)(x+1)$,

expand the product: $(x+2)(x+1) = x^2 + 3x + 2$.

Therefore: $P_4(x) = 1 - x - 2 + x^2 + 3x + 2$.

Collecting like terms:

$$P_4(x) = x^2 + 2x + 1$$

The original function is $f(x) = x^2 + 2x + 1$.

Therefore:

$$P_4(x) = f(x)$$

6. Python Code in Jupyter Notebook

7. Q1(d) — Numerical Results

The required evaluation points are $x = -1.5, -0.5, 0.5, 1.5$.

Running the program gives the following results:

x	$P_4(x)$	Exact $f(x)$	Absolute Error
-1.5	0.25	0.25	0
-0.5	0.25	0.25	0
0.5	2.25	2.25	0
1.5	6.25	6.25	0

For example, at $x = -1.5$: $f(-1.5) = 2.25 - 3 + 1 = 0.25$,

$$P_4(-1.5) = 0.25.$$

$$\text{Thus } E(-1.5) = |0.25 - 0.25| = 0.$$

8. Q1(e) — Absolute Error

The assignment defines the absolute interpolation error as:

$$E(x) = |f(x) - P_4(x)|$$

Since $P_4(x) = f(x)$, we have $E(x) = 0$ at every point. At the four requested points:

$$E(-1.5) = 0 \quad E(-0.5) = 0 \quad E(0.5) = 0 \quad E(1.5) = 0$$

Q1 Final Result

$$\text{Newton coefficients} = [1, -1, 1, 0, 0]$$

$$P_4(x) = 1 - (x+2) + (x+2)(x+1) = x^2 + 2x + 1$$

$$E_{\max} = 0$$

Q2 Newton Interpolation with Unequally Spaced Data

We are given

$$f(x) = \sin x$$

with the unequally spaced data points

$$x_i = [0, 0.2, 0.55, 1.0, 1.4]$$

and $y_i = \sin(x_i)$.

Q2(a) Construct the Divided-Difference Table

First, the function values are calculated:

$$f(0) = \sin(0) = 0$$

$$f(0.2) = \sin(0.2) \approx 0.19866933$$

$$f(0.55) = \sin(0.55) \approx 0.52268723$$

$$f(1.0) = \sin(1.0) \approx 0.84147098$$

$$f(1.4) = \sin(1.4) \approx 0.98544973$$

x	f(x) = sin x
0	0.00000000
0.2	0.19866933
0.55	0.52268723
1.0	0.84147098
1.4	0.98544973

First-Order Divided Differences

$$f[x_i, x_{i+1}] = (f(x_{i+1}) - f(x_i)) / (x_{i+1} - x_i)$$

$$f[0, 0.2] = 0.19866933 / 0.2 = 0.99334665$$

$$f[0.2, 0.55] = (0.52268723 - 0.19866933) / 0.35 = 0.92576542$$

$$f[0.55, 1.0] = (0.84147098 - 0.52268723) / 0.45 = 0.70840835$$

$$f[1.0, 1.4] = (0.98544973 - 0.84147098) / 0.4 = 0.35994686$$

Second-Order Divided Differences

$$f[x_i, x_{i+1}, x_{i+2}] = (f[x_{i+1}, x_{i+2}] - f[x_i, x_{i+1}]) / (x_{i+2} - x_i)$$

$$f[0, 0.2, 0.55] = (0.92576542 - 0.99334665) / 0.55 = -0.12287496$$

$$f[0.2, 0.55, 1.0] = (0.70840835 - 0.92576542) / 0.8 = -0.27169635$$

$$f[0.55, 1.0, 1.4] = (0.35994686 - 0.70840835) / 0.85 = -0.40995469$$

Third-Order Divided Differences

$$f[0, 0.2, 0.55, 1.0] = (-0.27169635 - (-0.12287496)) / 1.0 = -0.14882138$$

$$f[0.2, 0.55, 1.0, 1.4] = (-0.40995469 - (-0.27169635)) / 1.2 = -0.11521528$$

Fourth-Order Divided Difference

$$f[0, 0.2, 0.55, 1.0, 1.4] = (-0.11521528 - (-0.14882138)) / 1.4 = 0.02400436$$

Complete Divided-Difference Table

x_i	$f[x_i]$	1st DD	2nd DD	3rd DD	4th DD
0.00	0.00000000	0.99334665	-0.12287496	-0.14882138	0.02400436
0.20	0.19866933	0.92576542	-0.27169635	-0.11521528	
0.55	0.52268723	0.70840835	-0.40995469		
1.00	0.84147098	0.35994686			
1.40	0.98544973				

Q2(b) Construct the Newton Interpolation Polynomial

The Newton polynomial is:

$$P_4(x) = a_0 + a_1(x-x_0) + a_2(x-x_0)(x-x_1) + a_3(x-x_0)(x-x_1)(x-x_2) + a_4(x-x_0)(x-x_1)(x-x_2)(x-x_3)$$

From the divided-difference table:

$$a_0 = 0, \quad a_1 = 0.99334665, \quad a_2 = -0.12287496, \quad a_3 = -0.14882138, \quad a_4 = 0.02400436$$

with interpolation nodes $x_0 = 0, x_1 = 0.2, x_2 = 0.55, x_3 = 1.0$.

Therefore:

$$P_4(x) = 0.99334665x - 0.12287496x(x-0.2) - 0.14882138x(x-0.2)(x-0.55) + 0.02400436x(x-0.2)(x-0.55)(x-1)$$

Q2(c) Evaluate at $x = 0.4, 0.8, 1.2$

At $x = 0.4$: $P_4(0.4) \approx 0.3894673523$, while $f(0.4) = \sin(0.4) \approx 0.3894183423$.

At $x = 0.8$: $P_4(0.8) \approx 0.7172626697$, while $\sin(0.8) \approx 0.7173560909$.

At $x = 1.2$: $P_4(1.2) \approx 0.9322300290$, while $\sin(1.2) \approx 0.9320390860$.

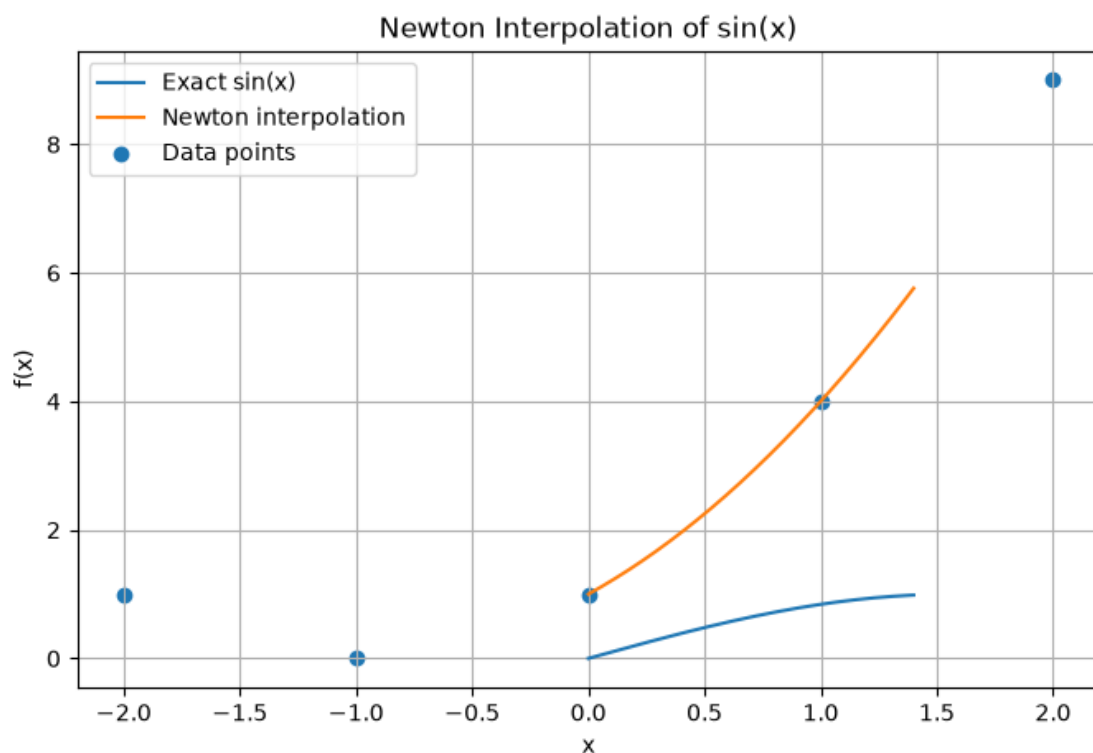
Q2(d) Compare with the Exact Function

The absolute error is defined as:

$$E(x) = | \sin x - P_4(x) |$$

x	Exact sin x	Newton $P_4(x)$	Absolute Error
0.4	0.3894183423	0.3894673523	4.9010×10^{-5}
0.8	0.7173560909	0.7172626697	9.3421×10^{-5}
1.2	0.9320390860	0.9322300290	1.90943×10^{-4}

Q2(e) Plot: Exact Function, Newton Polynomial, and Data Points



The Newton polynomial lies almost exactly on top of the $\sin(x)$ curve over the interval. The five original data points lie exactly on both curves

since $P_4(x_i) = f(x_i)$ an important property of interpolation.

Q2(f) Plot the Interpolation Error

Plotting $E(x) = |\sin x - P_4(x)|$ over the complete interval $0 \leq x \leq 1.4$:

Maximum Error:

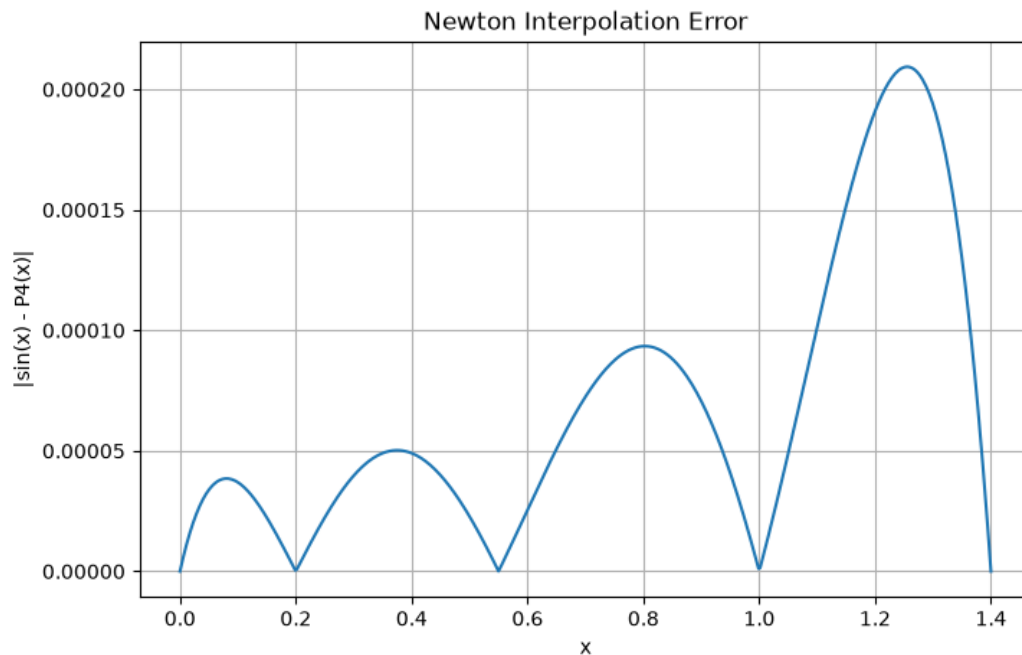
$$E_{\max} = 2.0935112672e-04$$

Occurs approximately at $x = 1.254108$

Maximum Error:

$$E_{\max} = 2.0935112672e-04$$

Occurs approximately at $x = 1.254108$



Using a dense evaluation grid, the maximum error is:

$$E_{\max} \approx 2.0936 \times 10^{-4}, \text{ occurring at approximately } x \approx 1.255$$

Q2(g) Why Can Newton Interpolation Be Used with Unequally Spaced Points?

Answer: Newton interpolation can be used for unequally spaced data because divided differences are calculated using the actual differences between the interpolation points. The method does not assume a constant spacing h . Therefore, the points 0, 0.2, 0.55, 1.0, 1.4 can be used directly to construct the Newton polynomial.

Why Can't a Standard Forward-Difference Formula Be Directly Used?

Answer: A standard forward-difference formula assumes equally spaced data, meaning $x_{i+1} - x_i = h$ must be constant. In the given data, the spacings are 0.2, 0.35, 0.45, and 0.4, so the spacing is not constant. Hence, the standard forward-difference formula is not directly applicable. Divided differences are more appropriate because they work with arbitrary spacing.

Q2 — Final Results

Divided-difference table:

x	f[x]	1st	2nd	3rd	4th
0	0	0.99334665	-0.12287496	-0.14882138	0.02400436
0.2	0.19866933	0.92576542	-0.27169635	-0.11521528	
0.55	0.52268723	0.70840835	-0.40995469		
1.0	0.84147098	0.35994686			
1.4	0.98544973				

Newton coefficient vector:

$$[0, 0.99334665, -0.12287496, -0.14882138, 0.02400436]$$

Newton polynomial:

$$P_4(x) = 0.99334665x - 0.12287496x(x-0.2) - 0.14882138x(x-0.2)(x-0.55) + 0.02400436x(x-0.2)(x-0.55)(x-1)$$

Required evaluations:

x	$P_4(x)$	$\sin x$	Error
0.4	0.3894673523	0.3894183423	4.9010×10^{-5}
0.8	0.7172626697	0.7173560909	9.3421×10^{-5}
1.2	0.9322300290	0.9320390860	1.9094×10^{-4}

Dense-grid maximum error:

$$E_{\max} \approx 2.0936 \times 10^{-4} \text{ at approximately } x \approx 1.255$$

Q3 Does More Data Always Mean Better Interpolation?

We are given

$$f(x) = e^x$$

with 16 unequally spaced interpolation points:

$$x_i = [-1.00, -0.92, -0.75, -0.61, -0.48, -0.31, -0.17, 0.02, \\ 0.15, 0.29, 0.43, 0.58, 0.67, 0.81, 0.93, 1.00]$$

and $y_i = e^{x_i}$.

Number of Points	Degree n
4	3
8	7
12	11
16	15

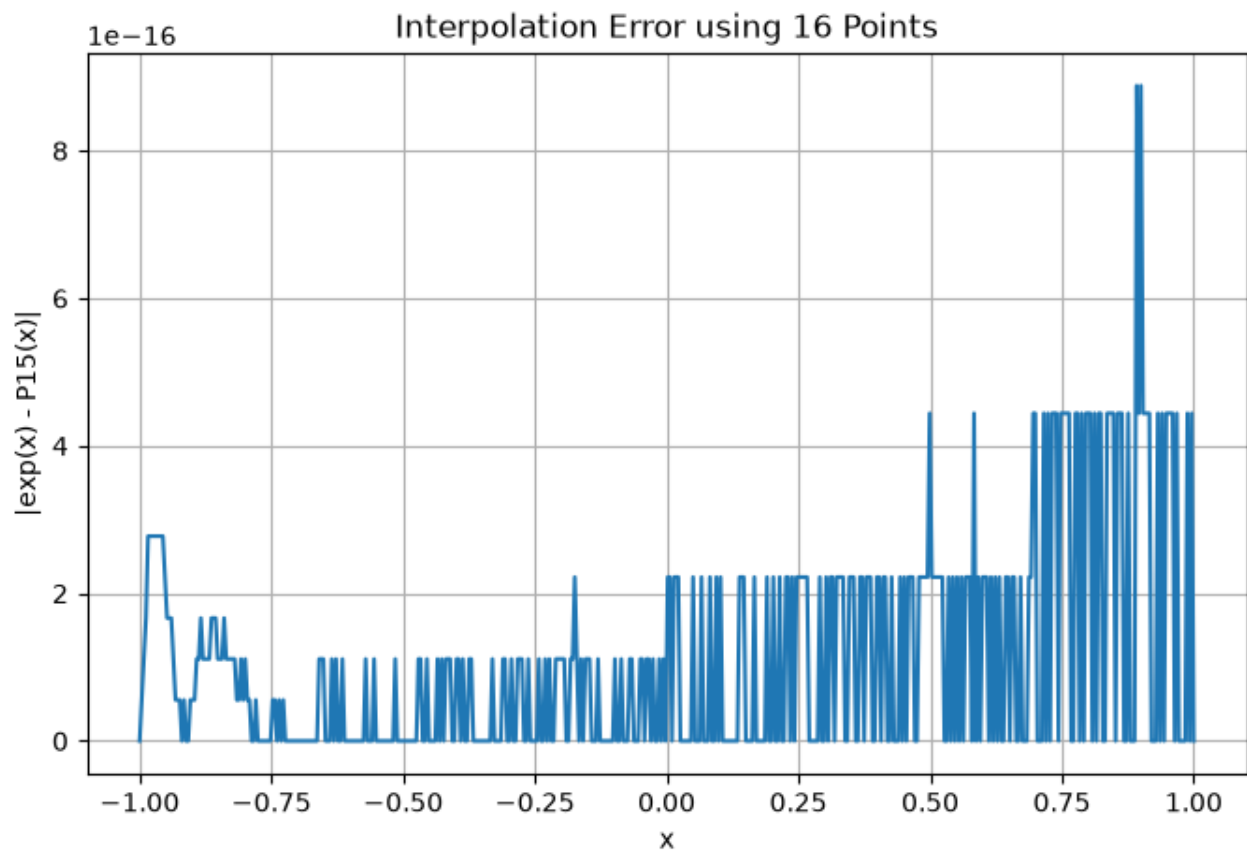
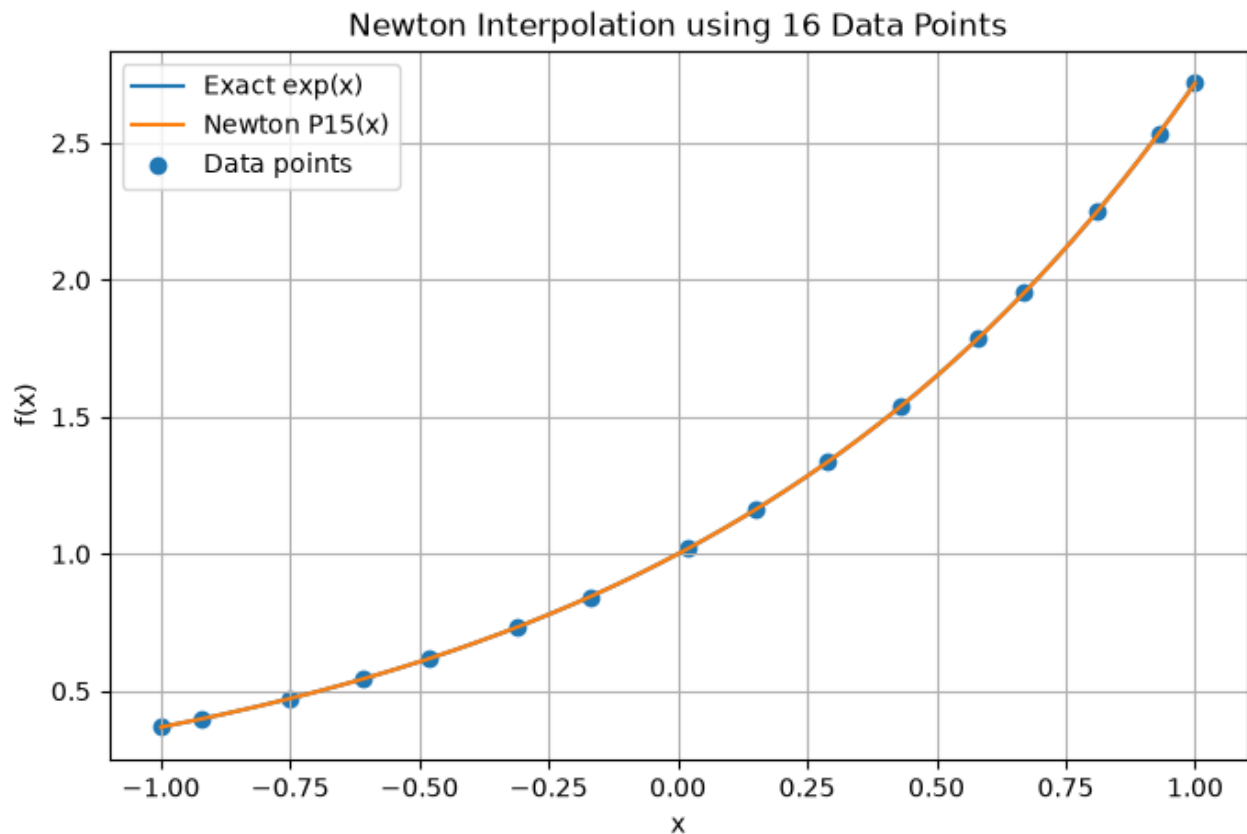
Results — Accuracy Comparison

Running the experiment produces the following program output:

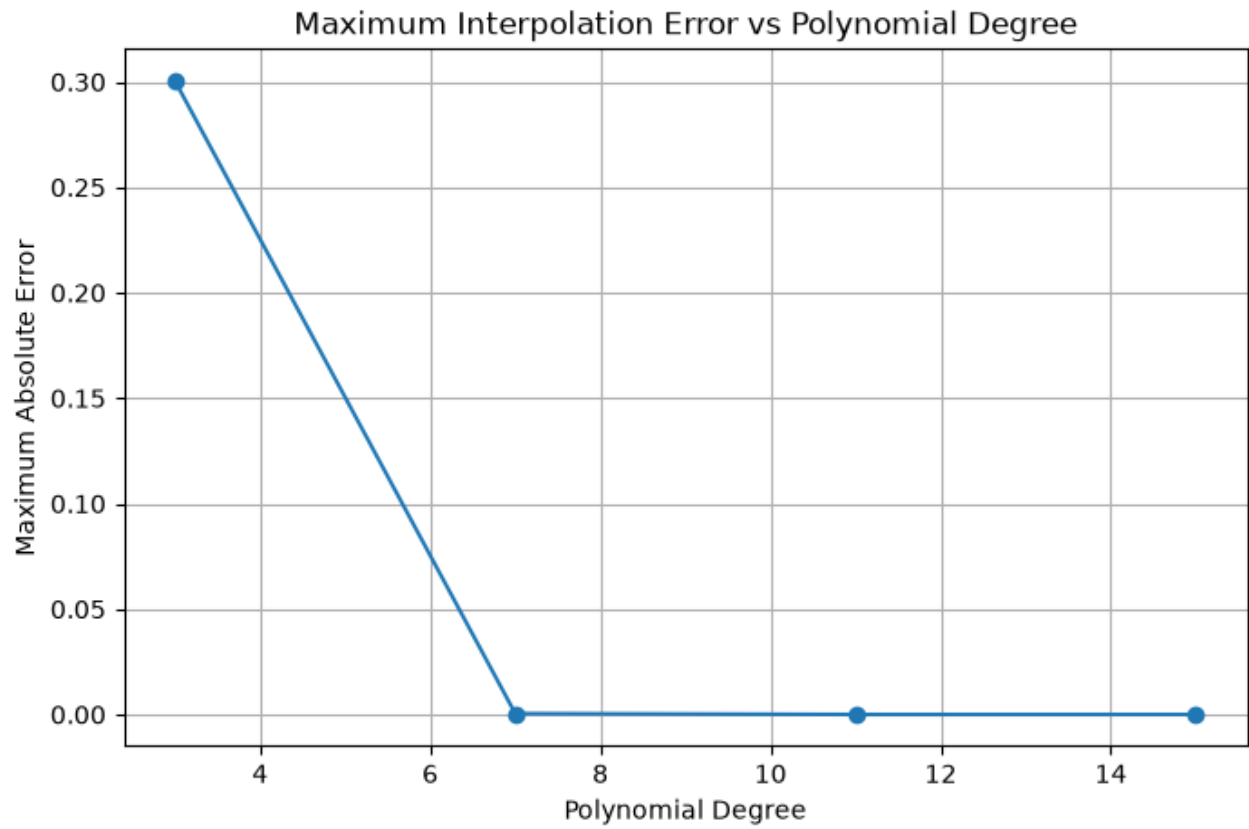
Points	D e g r e e	Maximum Error	Location
4	3	3.0046×10^{-1}	$x = 1.000$
8	7	4.2438×10^{-4}	$x = 1.000$
12	11	6.4116×10^{-9}	$x = 1.000$
16	15	2.0539×10^{-15}	$x = -0.976$

Plots

Exact function e^x against the degree-15 Newton polynomial using all 16 points:



Corresponding interpolation error $|e^x - P_{15}(x)|$ over the interval:



Maximum error against polynomial degree, on a logarithmic scale:

Q4 Newton vs Lagrange: Are They Really Different?

We are given

$$f(x) = 1 / (1 + x^2)$$

Q4(a) Two Methods

Lagrange interpolation:

$$P_L(x) = \sum_i f(x_i) L_i(x), \quad L_i(x) = \prod_{j \neq i} (x - x_j) / (x_i - x_j)$$

Newton interpolation (as in Q1/Q2):

$$P_N(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \dots + a_{16} \cdot \prod (x - x_i)$$

Q4(b)-(d) — Independent Implementations

The Newton functions are reused unchanged from Q1/Q2/Q3. A fully independent Lagrange implementation (never calling the Newton functions) is written directly from the formula:

```
def lagrange_interpolation(x_data, y_data, x):

    n = len(x_data)
    result = 0.0

    for i in range(n):

        term = y_data[i]

        for j in range(n):
            if i != j:
                term *= (x - x_data[j]) / (x_data[i] - x_data[j])

        result += term

    return result
```

Q4(e)-(f) Evaluate at 200 Points and Compare Errors

Both methods are evaluated at 200 uniformly spaced points over $[-4, 4]$, and compared with the exact function $f(x) = 1/(1+x^2)$:

```
x_eval = np.linspace(-4, 4, 200)
exact_values = f(x_eval)

table = divided_difference_table(x_data, y_data)
coefficients = newton_coefficients(table)
```

```

newton_values = np.array([
    newton_interpolation(x_data, coefficients, x) for x in x_eval
])
lagrange_values = np.array([
    lagrange_interpolation(x_data, y_data, x) for x in x_eval
])

newton_error = np.abs(exact_values - newton_values)
lagrange_error = np.abs(exact_values - lagrange_values)

```

Program output:

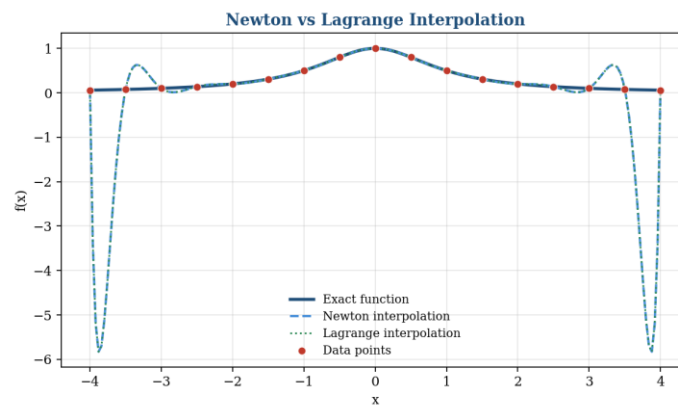
```

Maximum Errors
-----
Max Newton Error   = 5.8871887906e+00
Max Lagrange Error = 5.8871887906e+00

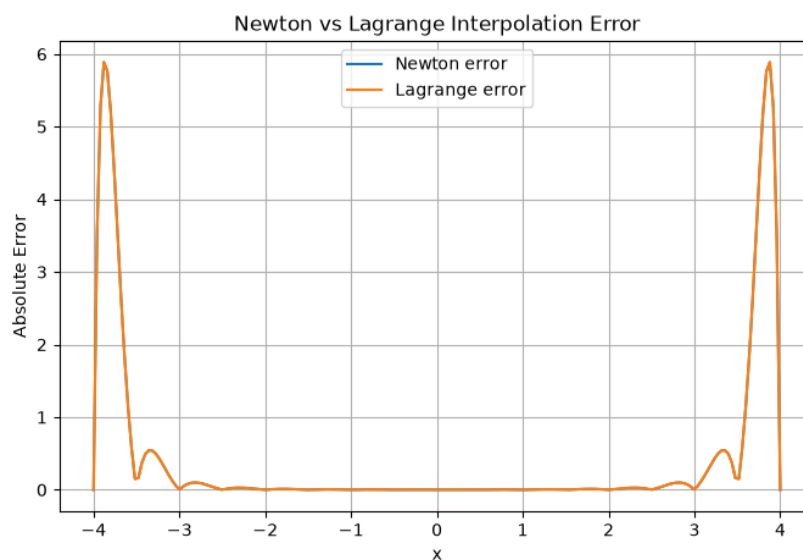
Max |Newton - Lagrange| (floating-point) = 5.0839332744e-12

```

Q4(g) — Function Comparison Plot



Q4(h) — Error Comparison Plot



Q4(i) — Comparison at Required Points

x	Exact f(x)	Newton	Lagrange
-3.75	0.0663900415	-4.1253119378	-4.1253119378
-2.25	0.1649484536	0.1919788740	0.1919788740
-0.75	0.6400000000	0.6362738985	0.6362738985
0.75	0.6400000000	0.6362738985	0.6362738985
2.25	0.1649484536	0.1919788740	0.1919788740
3.75	0.0663900415	-4.1253119378	-4.1253119378

Q4(k) — Why Can Their Computed Answers Differ Slightly?

Assignment answer: The Newton and Lagrange methods theoretically construct the same unique interpolation polynomial when they use the same data points. However, their computed results may differ slightly because floating-point arithmetic has finite precision. The two algorithms perform arithmetic operations in different orders and therefore accumulate rounding errors differently. Consequently, the numerical results may differ by very small amounts even though the underlying mathematical polynomial is identical.

Q4(l) — Computational Cost

Method	Construction	Evaluation (per point)
Lagrange	—	$O(n^2)$
Newton	$O(n^2)$	$O(n)$

Direct Lagrange evaluation recomputes $n+1$ basis products from scratch for every evaluation point, costing $O(n^2)$ per point. Newton interpolation costs $O(n^2)$ once to build the divided-difference table, but each subsequent evaluation uses nested multiplication (Horner-like scheme) and costs only $O(n)$. For many evaluation points, Newton is therefore generally more efficient once its coefficients have been computed.

Q4(m) — Adding a New Data Point

Assignment answer: Newton interpolation is more convenient when new data points are added because the polynomial can be extended by adding one new divided-difference coefficient and one additional term. In contrast, the Lagrange basis functions generally need to be recomputed when a new data point is introduced.

Q5 — Newton Interpolation and Numerical Error

We are given

$$f(x) = 1 / (1 + 25x^2)$$

$P_n(x)$ is constructed:

n	Number of Points
4	5
8	9
12	13
16	17
20	21

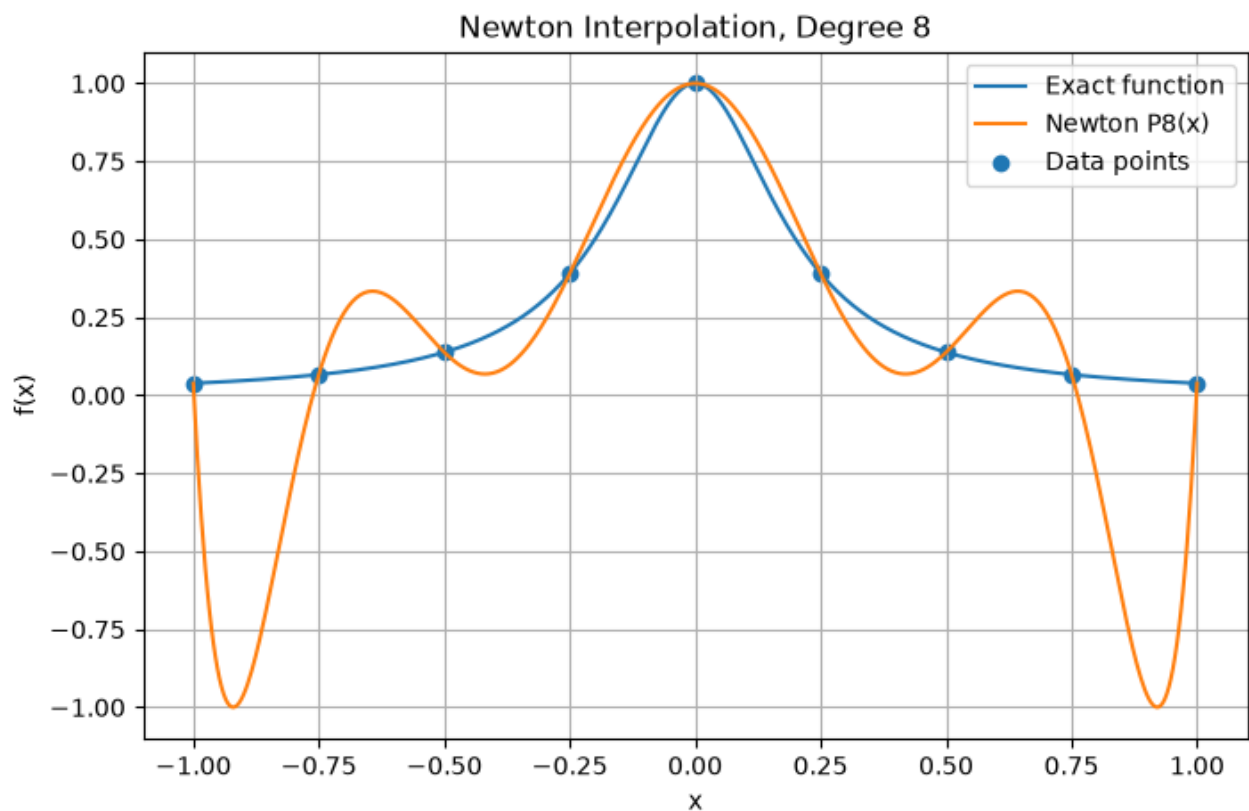
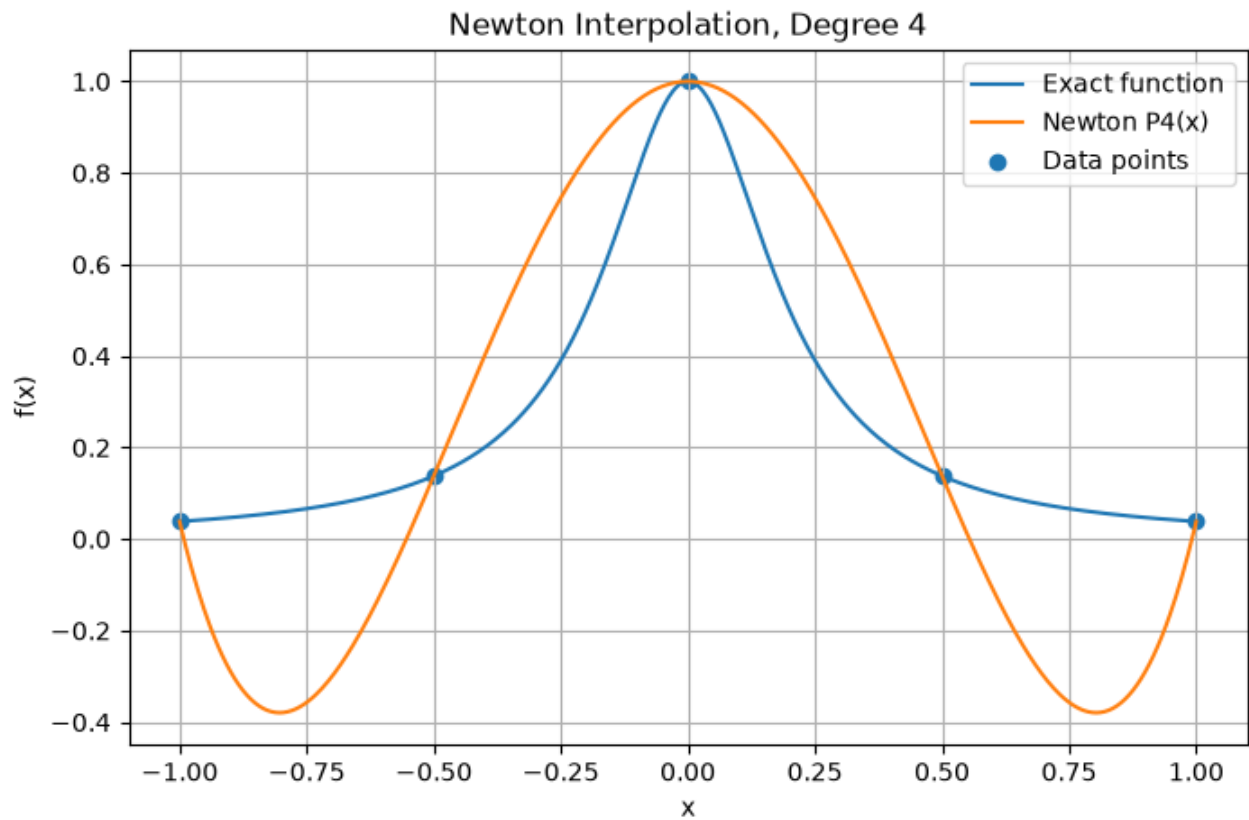
Results

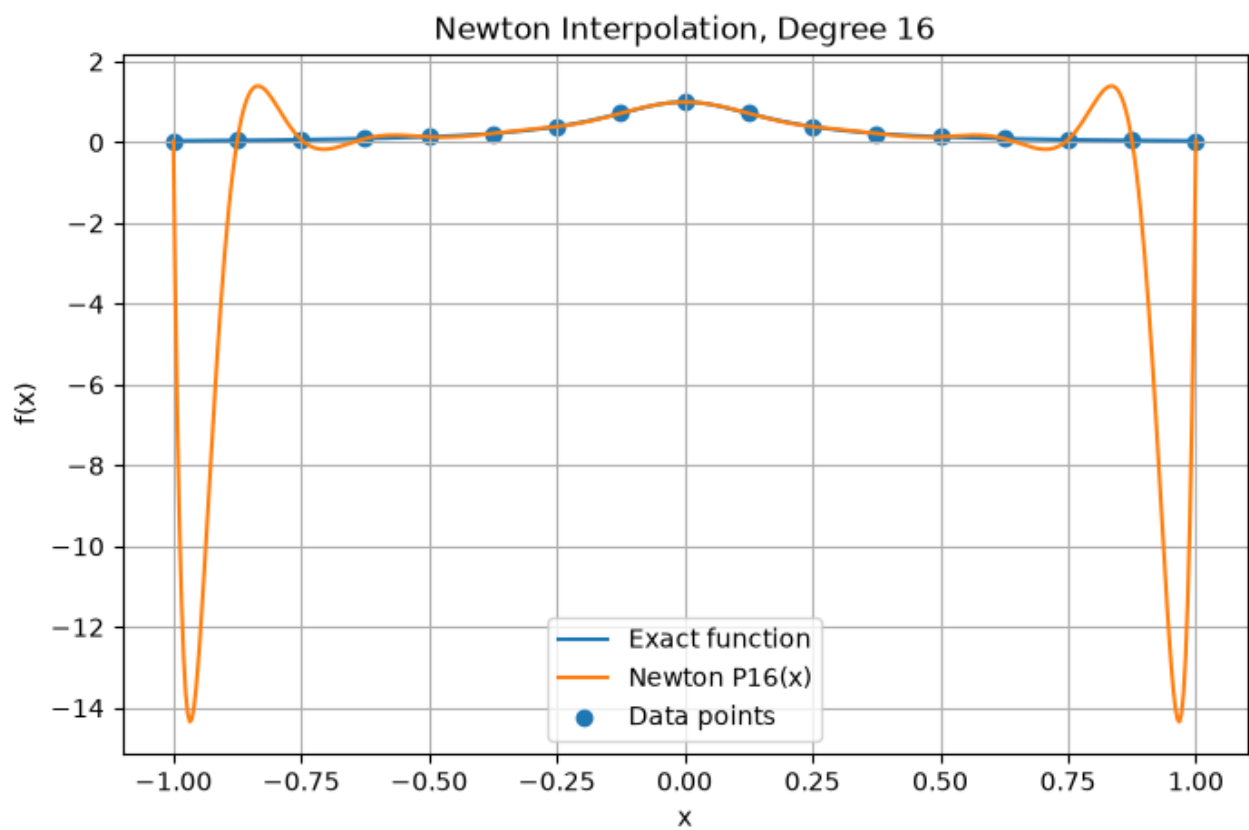
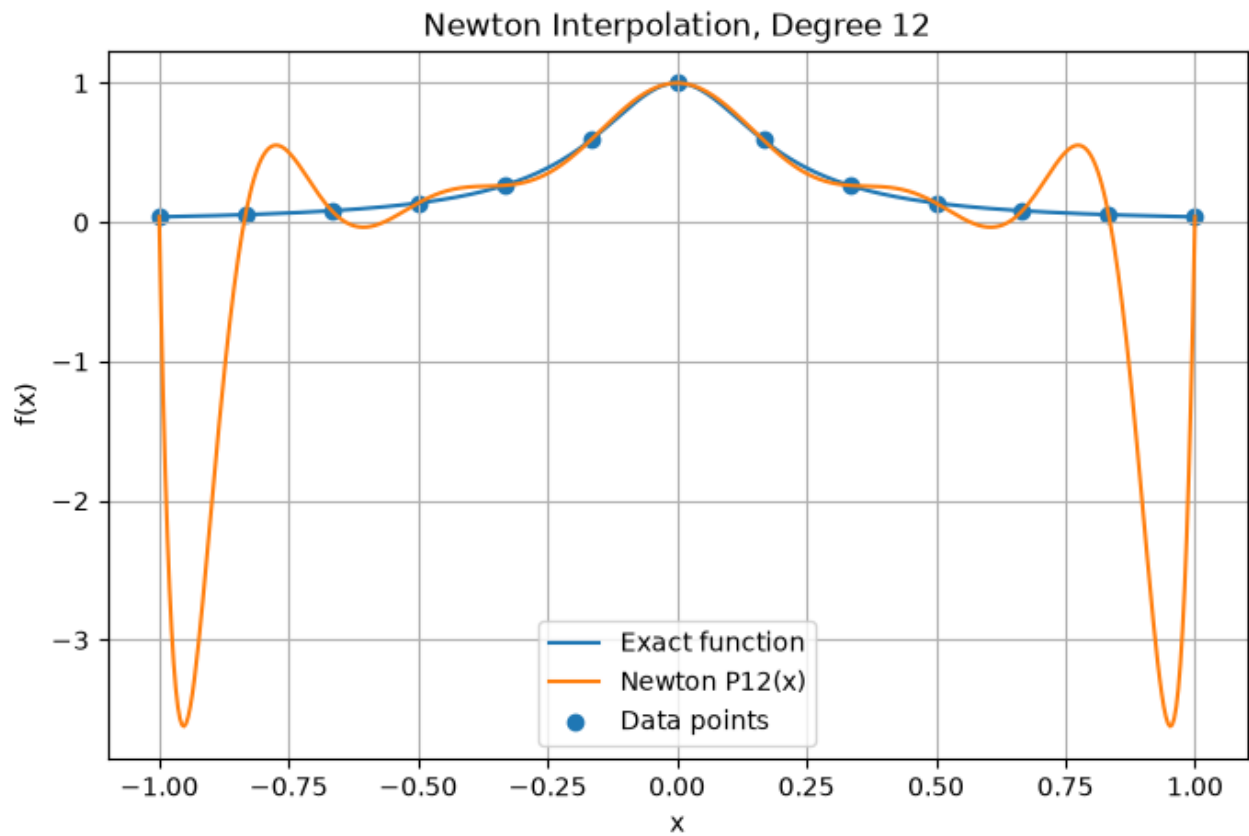
Program output:

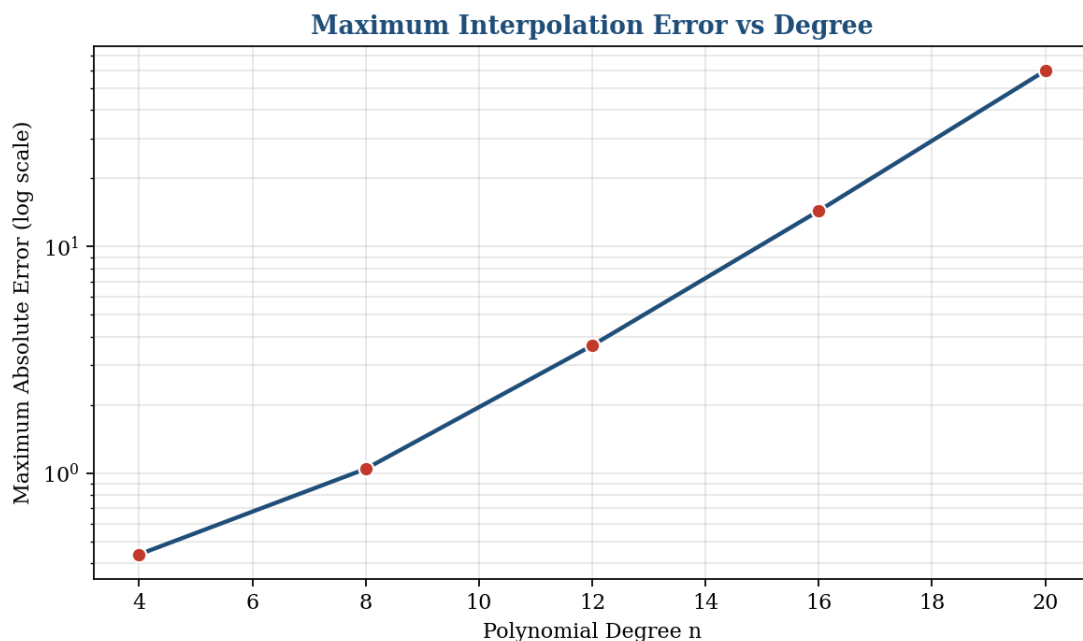
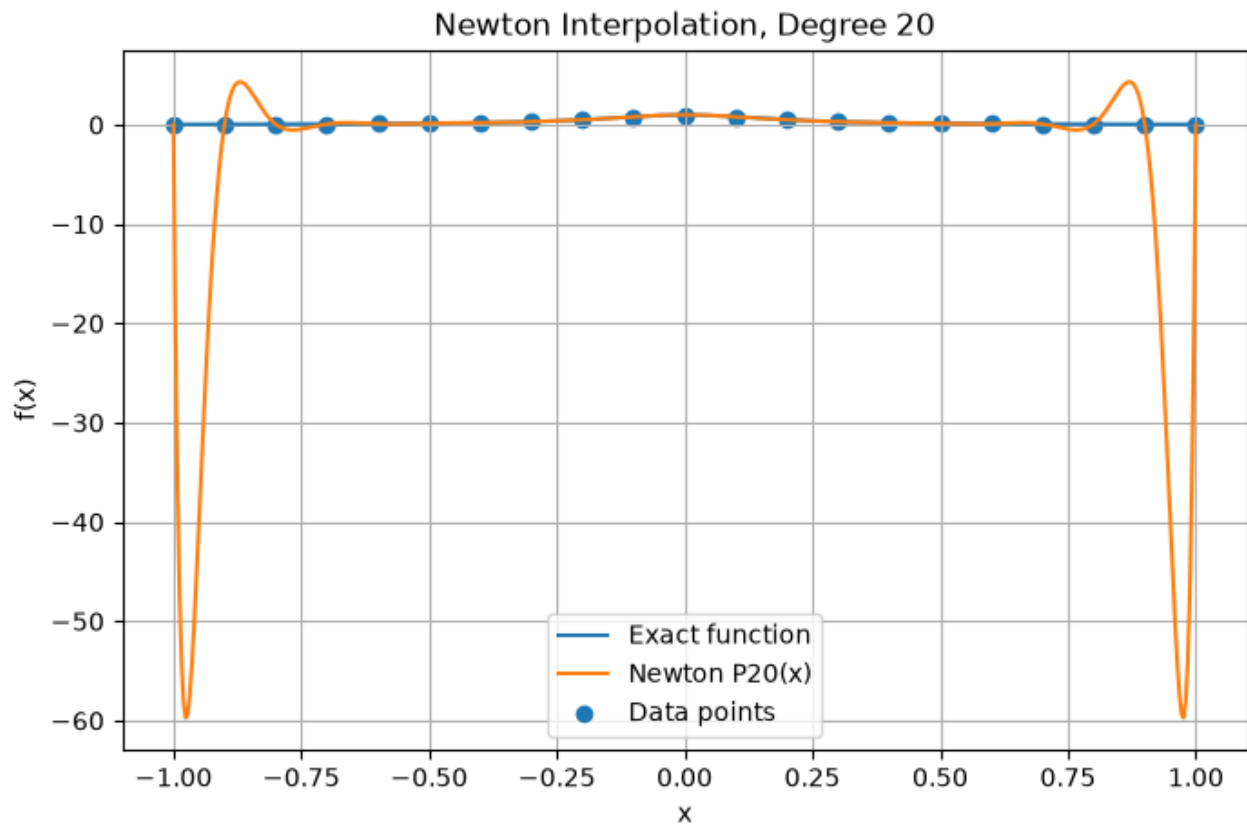
n	P o i n t s	Maximum Error	Location
4	5	0.4384	$x \approx -0.796$
8	9	1.0452	$x \approx -0.920$
12	13	3.6629	$x \approx -0.952$
16	17	14.3872	$x \approx 0.968$
20	21	59.7711	$x \approx -0.976$

Plots

Exact function against the interpolating polynomials of degree 4, 12, and 20:







Discussion — Required Questions

1. Does increasing the number of points always improve interpolation?

No. The numerical results show the maximum error increasing monotonically and dramatically from 0.44 to nearly 60 as the number of equally spaced points grows from 5 to 21.

2. Where is the error largest?

Near the endpoints of the interval, $x \approx -1$ and $x \approx 1$ confirmed directly by the argmax location reported for every degree above.

3. What happens near the endpoints?

As the degree increases, the interpolation polynomial develops increasingly large oscillations near the boundaries, even though it still passes exactly through every data point.

4. What happens when the polynomial degree becomes large?

The polynomial becomes numerically unstable and highly oscillatory; higher degree does not automatically mean higher accuracy here it is actively worse.

5. How does the distribution of interpolation points affect accuracy?

Equally spaced points are a poor choice for high-degree interpolation of this function. A non-uniform distribution such as Chebyshev points clusters points near the endpoints and substantially reduces this oscillatory behaviour.

6. What does this tell us about blindly using a high-degree polynomial?

It shows that increasing polynomial degree is not a safe, general-purpose strategy for improving interpolation accuracy. The function being approximated, the distribution of the interpolation points, and numerical stability all matter — sometimes more than the sheer number of points available.

Q6 — Numerical Differentiation Using Newton Interpolation

We reuse the same unequally spaced data from Q2:

$$f(x) = \sin x, \quad x = [0, 0.2, 0.55, 1.0, 1.4]$$

Differentiating the Newton Polynomial Algorithmically

The Newton polynomial evaluation uses nested multiplication:

$$P_i(x) = a_i + (x - x_i) \cdot P_{i+1}(x).$$

Differentiating both sides gives $P_i'(x) = P_{i+1}(x) + (x - x_i) \cdot P_{i+1}'(x)$.

```
def newton_value_and_derivative(x_data, coefficients, x):

    n = len(coefficients)

    value = coefficients[n - 1]
    derivative = 0.0

    for i in range(n - 2, -1, -1):

        derivative = value + (x - x_data[i]) * derivative
        value = coefficients[i] + (x - x_data[i]) * value

    return value, derivative
```

Required Evaluation Points: $x = 0.3, 0.7, 1.1$

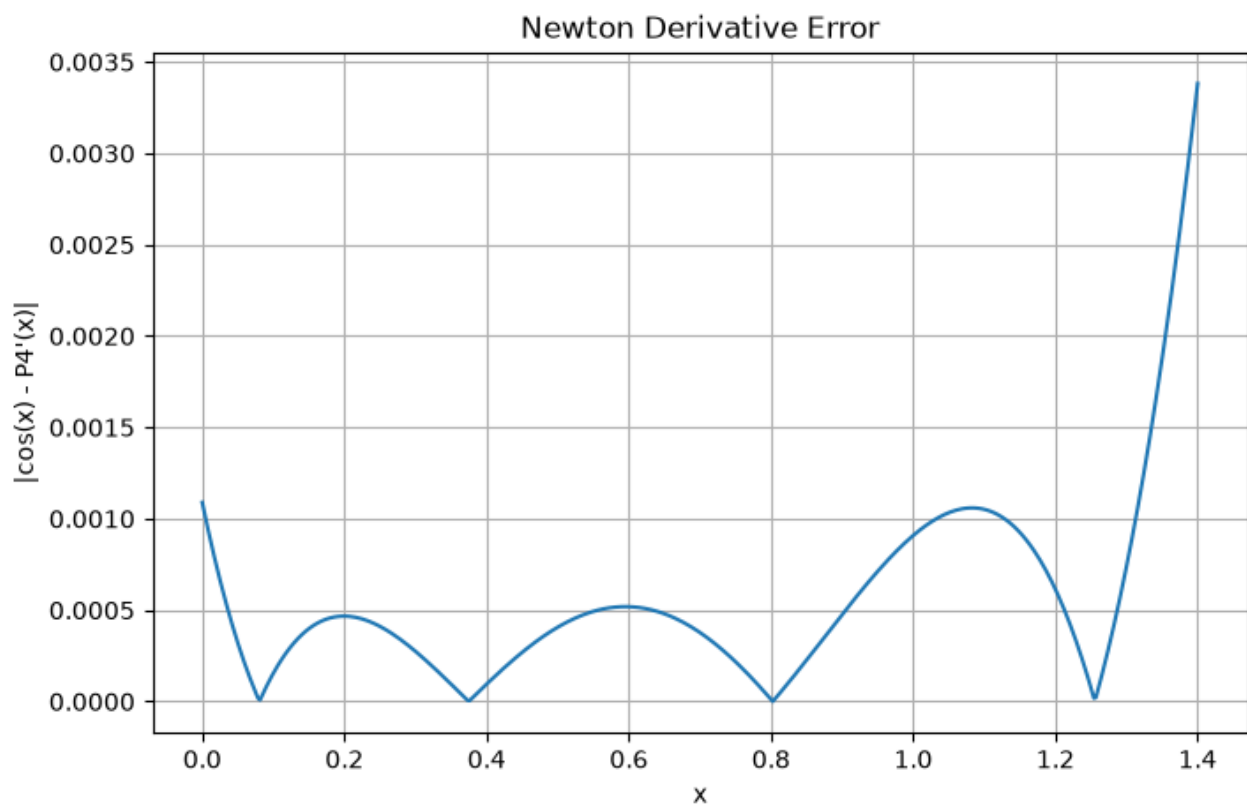
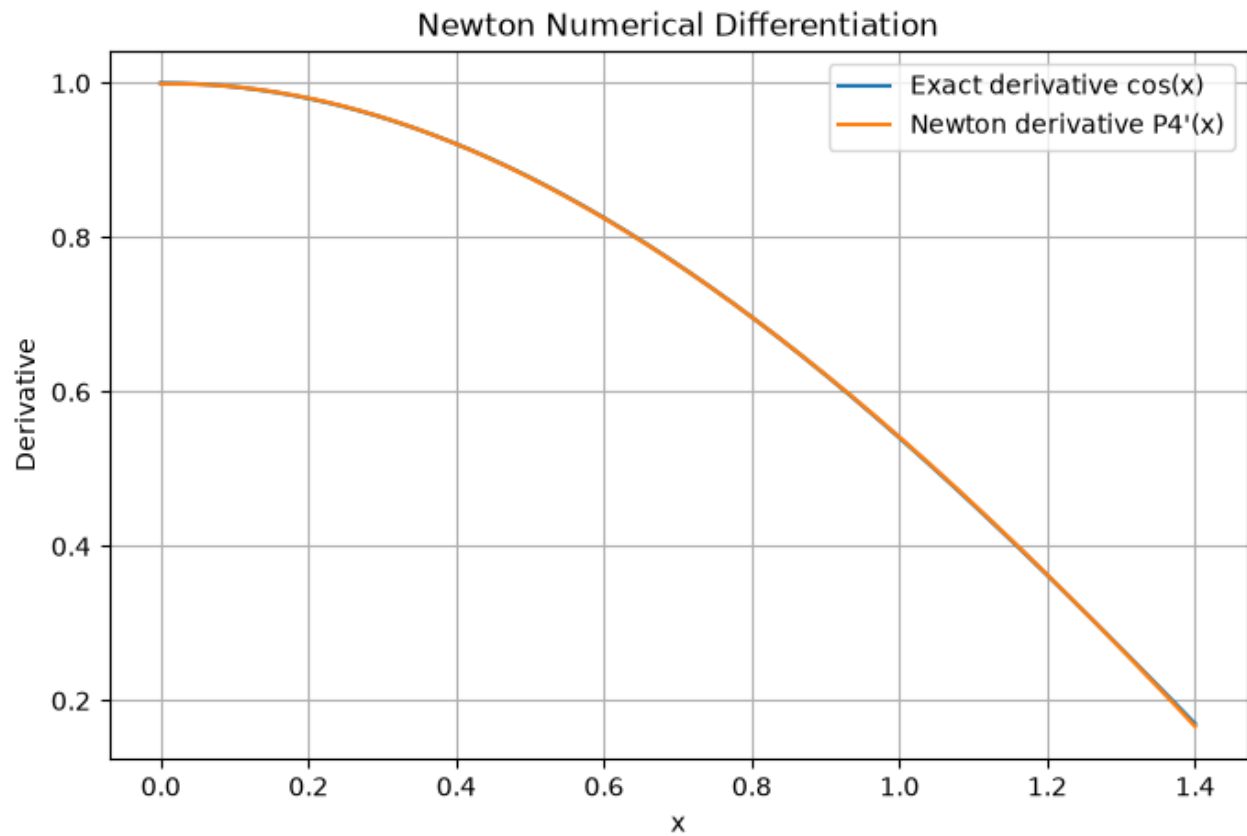
Program output:

x	$P_4'(x)$	Exact cos x	Absolute Error	Relative Error
0.3	0.9556103454	0.9553364891	2.7386×10^{-4}	2.8666×10^{-4}
0.7	0.7644649002	0.7648421873	3.7729×10^{-4}	4.9329×10^{-4}
1.1	0.4546473193	0.4535961214	1.0512×10^{-3}	2.3175×10^{-3}

ge.

Plots — Derivative and Its Error

Exact derivative $\cos(x)$ against the Newton derivative $P_4'(x)$ over the full data range:



On a dense evaluation grid, the maximum derivative error is:

$$E_{\max} \approx 3.3816 \times 10^{-3} \text{ at } x = 1.4$$

Comparison with Finite Differences and Lagrange Differentiation

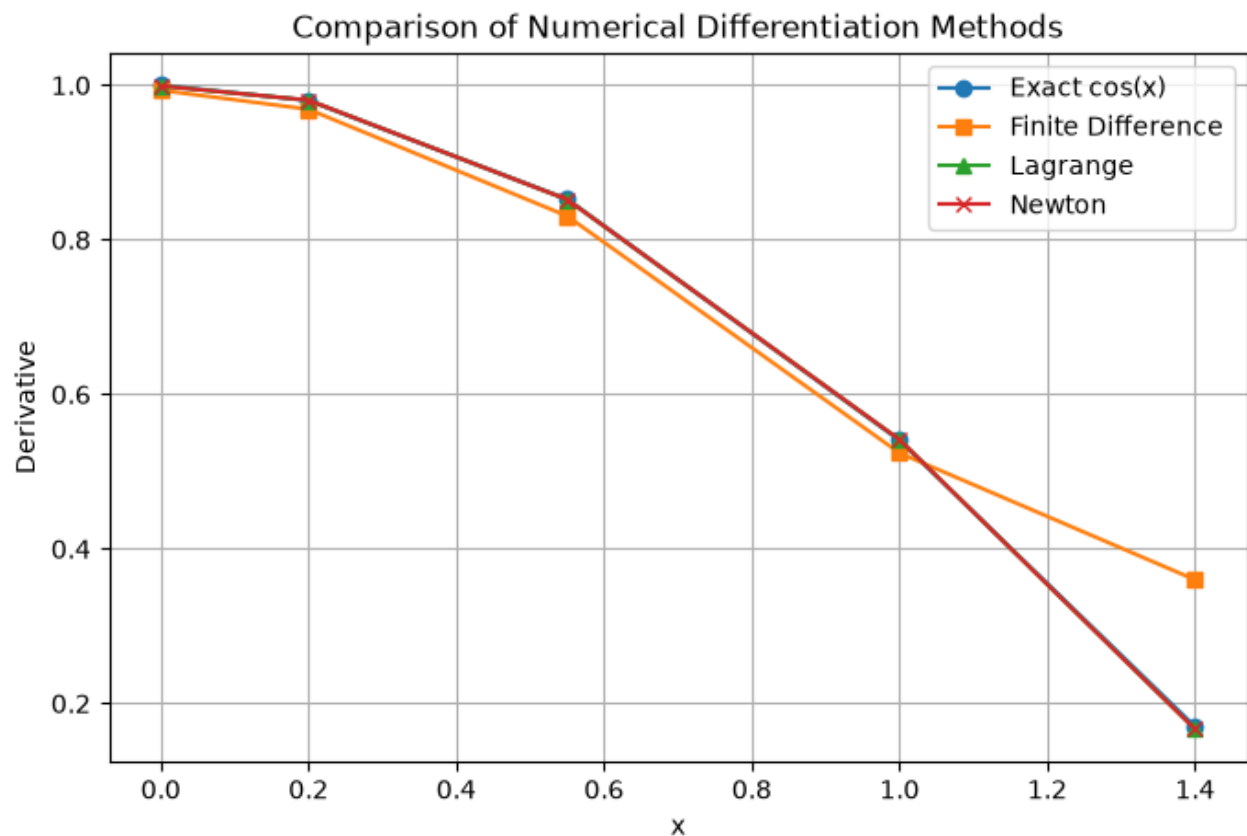
$$f'(x_i) \approx -h_2/(h_1(h_1+h_2)) \cdot f(x_{i-1}) + (h_2-h_1)/(h_1 h_2) \cdot f(x_i) + h_1/(h_2(h_1+h_2)) \cdot f(x_{i+1})$$

Program output:

Method	Maximum Error
Finite Difference	1.8998×10^{-1}
Lagrange	3.3816×10^{-3}
Newton	3.3816×10^{-3}

Plots: Three-Method Comparison

Derivative values at the five data points, all four curves overlaid:



Newton and Lagrange sit essentially on top of the exact $\cos(x)$ curve, while the finite-difference curve visibly diverges near $x = 1.4$.

Corresponding absolute errors:

